

實作導向學習手冊

# Git 版本控制 實作課程

---

從本機版本管理，到 GitHub 團隊協作

**你將完成：**初始化版本庫、提交、分支、合併、衝突處理、遠端同步、Pull Request、標籤、復原、rebase 與綜合專題。

適合初學者 / Windows PowerShell 實作 / 繁體中文版

## 如何使用這本書

這不是一本只介紹指令的手冊，而是一套可以逐步操作的課程。每課先建立觀念，再提供指令、預期結果、常見錯誤與實作任務。建議在測試版本庫親手輸入每個指令，並養成先看 `git status`、再決定下一步的習慣。

**核心循環：**修改 → `git diff` → `git add` → `git diff --staged` → `git commit` → `git push`

## 學習地圖

| 課 | 主題           | 成果                                                                                                    |
|---|--------------|-------------------------------------------------------------------------------------------------------|
| 1 | 版本控制與 Git 基礎 | 安裝、設定、三個區域                                                                                            |
| 2 | 提交與歷史        | <code>status</code> 、 <code>diff</code> 、 <code>commit</code> 、 <code>log</code>                      |
| 3 | 分支與合併        | <code>switch</code> 、 <code>branch</code> 、 <code>merge</code>                                        |
| 4 | 合併衝突         | 衝突標記與解決流程                                                                                             |
| 5 | GitHub 遠端版本庫 | <code>remote</code> 、 <code>push</code> 、 <code>fetch</code> 、 <code>pull</code> 、 <code>clone</code> |
| 6 | 團隊協作         | 功能分支、Pull Request、Review                                                                              |
| 7 | 進階日常操作       | <code>ignore</code> 、 <code>stash</code> 、 <code>tag</code> 、復原                                       |
| 8 | 歷史整理與實作      | <code>rebase</code> 與期末專題                                                                             |

## 第 1 課

# 版本控制與 Git 基礎

理解 Git 解決什麼問題，建立第一個本機版本庫

## 1.1 什麼是版本控制？

版本控制是一套記錄檔案變更歷史的機制。它讓你知道誰在何時改了什麼、為什麼修改，也讓你能比較版本、還原錯誤並支援多人協作。

| 沒有版本控制                     | 使用 Git                     |
|----------------------------|----------------------------|
| 檔名變成 final、final2、真的 final | 每次 commit 都是可辨識的存檔點        |
| 難以比較內容                     | git diff 精確顯示逐行差異          |
| 多人修改容易覆蓋                   | 分支與合併保留各自工作                |
| 出錯不易回復                     | restore、revert、reflog 提供復原 |

## 1.2 Git 與 GitHub 的差別

| 項目 | Git         | GitHub                  |
|----|-------------|-------------------------|
| 性質 | 本機版本控制工具    | 線上 Git 託管與協作平台          |
| 網路 | 離線也能使用      | push、pull、PR 通常需要網路     |
| 功能 | 提交、分支、合併、復原 | 備份、分享、PR、Issues、Actions |
| 安裝 | 需要安裝        | 網站不用安裝；Desktop/CLI 可選   |

**重點：**git push 需要 Git 與遠端版本庫，但不需要安裝 GitHub Desktop。

## 1.3 安裝與設定

```
git --version
git config --global user.name "你的名字"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global --list
```

## 1.4 建立第一個版本庫

```
mkdir git-practice
cd git-practice
git init
git status
```

## 1.5 Git 的三個區域

| 區域    | 用途                 | 移動方式       |
|-------|--------------------|------------|
| 工作目錄  | 正在編輯的實際檔案          | 修改檔案       |
| 暫存區   | 選擇下一次 commit 的內容快照 | git add    |
| 本機版本庫 | 保存正式版本歷史           | git commit |

```
工作目錄 --git add--> 暫存區 --git commit--> 本機版本庫
```

## 第 2 課

# 建立提交與閱讀歷史

掌握每天最常使用的 Git 工作循環

## 2.1 status、diff、add、commit

```
git status
git diff
git add README.md
git diff --staged
git commit -m "新增專案說明"
```

| 指令                | 回答的問題            |
|-------------------|------------------|
| git status        | 目前分支與各區域狀態       |
| git diff          | 尚未暫存的逐行差異        |
| git diff --staged | 下一次 commit 會包含什麼 |
| git add <file>    | 把檔案當下快照加入暫存區     |
| git commit -m     | 保存暫存區為正式版本       |

**關鍵觀念：**git add 保存的是當下快照。同一檔案在 add 後繼續修改，可能同時出現在已暫存與未暫存區。

## 2.2 查看歷史

HEAD 表示目前所在位置；每個 commit ID 都是內容與歷史關係產生的唯一識別碼。

```
git log
git log --oneline
git log --oneline --graph --decorate --all
git show <commit-ID>
git show --stat HEAD
```

## 2.3 修改最近一次提交

```
git commit --amend -m "新的提交訊息"
git add 漏掉的檔案
git commit --amend --no-edit
```

**警告：**amend 會產生新的 commit ID。只在尚未推送或已與協作者協調時使用。

## 2.4 好的提交訊息

- 用動詞開頭，並讓一個 commit 只處理一個清楚目的。
- 避免『修改內容』『update』等模糊訊息；提交前用 `git diff --staged` 再檢查一次。

**實作：**建立 `notes.md` 與 `todo.md`，依照不同目的分成兩個 commit，並用 `git log --oneline` 檢查訊息。

## 第 3 課

# 分支與合併

在不影響 main 的情況下安全開發新功能

## 3.1 分支概念

分支是指向某個 commit 的可移動名稱。main 只是常用的主要分支名稱；功能分支讓新工作與穩定版本分開。

```
git branch
git switch -c feature/add-guide
git switch main
git merge feature/add-guide
git branch -d feature/add-guide
```

## 3.2 正確合併方向

- 確認功能分支已 commit，工作目錄乾淨。
- 切換到接收變更的 main。
- 執行 `git merge <功能分支>`。
- 測試合併結果。
- 用 `git branch -d` 刪除已完成分支。

**口訣：**要把功能合併到哪裡，就先切換到哪個分支。

## 3.3 Fast-forward 與 Merge commit

| 類型           | 何時發生               | 結果                 |
|--------------|--------------------|--------------------|
| Fast-forward | 主線在分支建立後沒有新 commit | 主線指標直接前進           |
| Merge commit | 兩條分支都各自前進          | 建立有兩個父提交的合併 commit |

## 第 4 課

# 處理合併衝突

理解衝突不是故障，而是需要人類做內容決策

## 4.1 衝突標記

兩個分支修改同一檔案的同一位置時，Git 無法自行判斷最終答案，會停止合併並寫入標記。

```
<<<<<<< HEAD
課程狀態：即將完成
=====
課程狀態：進行中
>>>>>> feature/update-progress
```

## 4.2 標準解決流程

```
git status
# 編輯衝突檔案
git add progress.txt
git commit -m "合併課程進度並解決衝突"
```

- `git status` 找出 both modified 檔案。
- 閱讀 HEAD 與另一分支內容。
- 決定最終內容並移除所有衝突標記。
- `git add <檔案>` 標記已解決。
- `git commit` 完成合併。

**取消合併：**尚未準備處理時，可執行 `git merge --abort` 回到合併前。

## 4.3 完成檢查

- 檔案內不再有衝突標記。
- `git status` 顯示 All conflicts fixed。
- 合併後功能可正常運作。
- 不要在未完成合併時任意切換分支。



## 第 5 課

# GitHub 遠端版本庫

連接本機與雲端，理解 push、fetch、pull 與 clone

## 5.1 設定 origin 與第一次推送

origin 是遠端版本庫的慣用別名。-u 會設定本機 main 追蹤 origin/main；以後通常直接使用 git push。

```
git remote add origin https://github.com/帳號/專案.git
git remote -v
git push -u origin main
```

**常見錯誤：**Repository not found 通常表示網址錯誤、尚未建立或沒有權限。GitHub 網址使用使用者名稱，不使用 Email。

## 5.2 push、fetch 與 pull

| 指令               | 方向              | 是否改工作檔案              |
|------------------|-----------------|----------------------|
| git push         | 本機 → GitHub     | 發布 commit            |
| git fetch origin | GitHub → 遠端追蹤指標 | 否                    |
| git pull         | GitHub → 目前分支   | 是；大致等於 fetch + merge |

## 5.3 Clone 既有專案

```
git clone https://github.com/帳號/專案.git
```

- 自動建立資料夾。
- 下載檔案與完整版本歷史。
- 自動設定 origin。
- 自動切出預設分支與追蹤關係。

**重點：**既有 GitHub 專案使用 clone，不需要再執行 git init 或 remote add origin。

## 第 6 課

# 團隊協作與 Pull Request

以功能分支提案、審查、合併並清理

## 6.1 標準協作流程

```
git switch -c feature/collaboration-note
git add collaboration.md
git commit -m "新增 Git 協作筆記"
git push -u origin feature/collaboration-note
```

- 從最新 main 建立功能分支。
- 修改、測試並建立目的清楚的 commit。
- 推送功能分支到 GitHub。
- 建立 PR 並檢查 Files changed。
- 完成 Review 與 Merge。
- 本機 fetch / pull 同步 main。
- 刪除本機及遠端功能分支。

## 6.2 Pull Request 的價值

- 合併前清楚呈現差異。
- 讓協作者討論設計與實作。
- 保留 Review、核准與合併紀錄。
- 可搭配自動測試與分支保護。

## 6.3 同步與清理

```
git fetch origin
git switch main
git pull
git branch -d feature/collaboration-note
git push origin --delete feature/collaboration-note
```

**安全刪除：**-d 只刪除已合併分支；-D 會強制刪除未合併分支。

## 第 7 課

## 進階日常操作與復原

忽略檔案、暫存半成品、標記版本，並選擇正確復原工具

## 7.1 .gitignore

.gitignore 只會直接忽略尚未追蹤的檔案。已提交過的檔案不會自動停止追蹤。

```
*.log
.env
node_modules/
dist/
.vscode/

git rm --cached tracked-file.txt
```

**敏感資料：**密碼或金鑰即使後來刪除，仍可能存在 Git 歷史中。最安全做法是一開始就不要提交。

## 7.2 git stash

```
git stash push -m "尚未完成的工作"
git stash list
git stash show -p "stash@{0}"
git stash apply "stash@{0}"
git stash drop "stash@{0}"
# 或：git stash pop
```

**提示：**若要連未追蹤檔案一起保存，可使用 `git stash -u`。

## 7.3 標籤 tag

分支會隨新提交前進；標籤通常固定指向發布時的 commit。

```
git tag -a v1.0.0 -m "完成第一版"
git show v1.0.0
git push origin v1.0.0
```

## 7.4 restore、revert、reset

| 工具      | 處理對象      | 改寫歷史 | 用途               |
|---------|-----------|------|------------------|
| restore | 未提交修改／暫存區 | 否    | 取消修改或取消暫存        |
| revert  | 已提交版本     | 否    | 以新 commit 撤銷共享歷史 |
| reset   | 本機分支位置    | 是    | 整理尚未共享的提交        |
| reflog  | HEAD 操作紀錄 | 否    | 尋找遺失的 commit     |

```
git restore README.md
git restore --staged README.md
git revert --no-edit <commit-ID>
git reset --soft HEAD~1
git reset HEAD~1
git reset --hard HEAD~1
git reflog
```

**警告：**reset --hard 會同步丟棄工作目錄修改。操作前確認分支、目標 commit 與狀態。

## 7.5 Reset 三種模式

| 模式           | 撤回 commit | 保留檔案修改 | 保留暫存 |
|--------------|-----------|--------|------|
| --soft       | 是         | 是      | 是    |
| --mixed (預設) | 是         | 是      | 否    |
| --hard       | 是         | 否      | 否    |

**救援：**誤 reset 後先用 git reflog 找 ID，再用 git branch rescue <ID> 建立救援分支。

## 第 8 課

# Rebase 與綜合實作

整理功能分支歷史，完成從開發到 Pull Request 的全流程

## 8.1 Rebase 的目的

Rebase 會暫時取下功能分支提交，把分支移到新的基礎，再逐一重新套用。結果更直線，但重新套用的 commit 會取得新 ID。

```
git switch feature/my-feature
git fetch origin
git rebase origin/main
```

| 情境          | 建議                         |
|-------------|----------------------------|
| 尚未推送的個人功能分支 | 可以 rebase，更新基礎並整理提交        |
| 已推送但只有自己使用  | 可評估；之後通常需 force-with-lease |
| 多人共同使用的分支   | 先協調，不要突然改寫他人基礎             |
| 共享 main     | 通常不要 rebase 已發布歷史          |

**安全推送：**更新已 rebase 的個人遠端分支時，優先用 `git push --force-with-lease`，而不是 `--force`。

## 8.2 Rebase 衝突

```
# 解決檔案衝突後
git add <file>
git rebase --continue

# 放棄整次 rebase
git rebase --abort
```

## 8.3 期末綜合專題

- 從最新 main 建立 feature/git-cheatsheet，並完成速查表。
- 用 diff 與 diff --staged 檢查後，建立目的清楚的 commit。
- 推送功能分支並建立 Pull Request。
- 在 GitHub Review、Merge。
- 本機切回 main、pull，並刪除本機與遠端功能分支。

**完成標準：**本機 main 與 origin/main 同步、沒有未提交變更、功能分支已清理、GitHub 保留 PR 紀錄。

## 附錄 A | Git 指令速查表

| 分類 | 指令                              | 用途         |
|----|---------------------------------|------------|
| 檢查 | git status                      | 查看分支與各區域狀態 |
| 比較 | git diff                        | 尚未暫存差異     |
| 比較 | git diff --staged               | 即將提交差異     |
| 暫存 | git add <file>                  | 加入內容快照     |
| 提交 | git commit -m "訊息"              | 建立正式版本     |
| 歷史 | git log --oneline --graph --all | 圖形化歷史      |
| 分支 | git switch -c <branch>          | 建立並切換分支    |
| 合併 | git merge <branch>              | 合併指定分支     |
| 遠端 | git fetch origin                | 取得資訊不合併    |
| 遠端 | git pull                        | 取得並合併      |
| 遠端 | git push                        | 發布提交       |
| 暫存 | git stash push -m "說明"          | 收起半成品      |
| 發布 | git tag -a v1.0.0 -m "說明"       | 建立標籤       |
| 復原 | git restore <file>              | 丟棄未暫存修改    |
| 復原 | git revert <commit>             | 安全撤銷提交     |
| 救援 | git reflog                      | 查看 HEAD 移動 |

## 附錄 B | 常見錯誤排查

| 訊息                          | 原因              | 處理方式                    |
|-----------------------------|-----------------|-------------------------|
| fatal: not a git repository | 目前不在含 .git 的資料夾 | 用 Get-Location 確認後進入版本庫 |
| Repository not found        | 網址錯誤、尚未建立或無權限   | 檢查 Code → HTTPS 網址與帳號   |
| non-fast-forward            | 遠端已有新 commit    | 先 fetch / pull 再處理差異    |
| branch is not fully merged  | 未被目前分支包含        | 切到正確分支，或確認後用 -D         |
| merge conflict              | 修改同一位置          | 編輯最終內容、add、commit       |
| rebase in progress          | rebase 尚未完成     | continue 或 abort        |

| 訊息                          | 原因           | 處理方式                                     |
|-----------------------------|--------------|------------------------------------------|
| LF will be replaced by CRLF | Windows 換行提醒 | 通常不是錯誤；可用 <code>.gitattributes</code> 統一 |

## 附錄 c | 日常安全檢查表

- 開始工作前：git status、git pull。
- 新功能：從最新 main 建立功能分支。
- 提交前：git diff、git diff --staged。
- 提交後：查看最近歷史並執行測試。
- 推送前：確認分支名稱與追蹤關係。
- 危險操作前：確認 commit，必要時建立備份分支。
- PR 合併後：pull main，刪除本機與遠端分支。

**最後口訣：**不確定時先執行 git status；遇到錯誤先讀訊息，不要急著使用 --force 或 --hard。

# 你已完成 Git 版本控制實作課程

下一步：把 Git 用在一個真實專案，每天完成小而清楚的提交。

```
git status → git add → git commit → git push
```